UT3 Bloque 1
Primera aplicación web y Servlets

Desarrollo Web en Entorno Servidor

De un programa que empieza en main a una aplicación Java que responde al navegador

Este bloque se aprende construyendo. Cada concepto aparece cuando hace falta para completar una acción: crear una ruta, recibir un dato, generar una respuesta o procesar un formulario. Al terminar podrás seguir una petición completa desde el navegador hasta el código Java y de vuelta al navegador.

> **Entorno de referencia.** Los ejemplos usan Java 17, Maven, Tomcat 10.1 y la API jakarta.servlet. Si el proyecto del aula tiene otro nombre, sustituye ut3-web por su context path real. En Tomcat 9 o anterior el paquete suele ser javax.servlet; no mezcles ambas familias.

Cómo trabajar con este bloque

No leas el documento de principio a fin sin ejecutar nada. En cada mini práctica escribe el código, despliega el proyecto, prueba la URL y compara el resultado con el esperado. Después reconstruye el recorrido de la información. Esa última parte convierte una receta en comprensión.

- Necesitas un JDK 17 configurado.
- Necesitas Apache Tomcat 10.1 añadido al entorno de desarrollo.
- El proyecto se llamará ut3-web en los ejemplos.
- La dirección habitual de Tomcat será http://localhost:8080.
- Cada cambio debe volver a compilarse y desplegarse antes de comprobarlo.

Ruta de aprendizaje

Parada	Lo que construirás	Idea central
1	HolaServlet	Una URL puede activar código Java.
2	SaludoServlet	La ruta pública no es el nombre de la clase.
3	Query string	La petición transporta parámetros.
4	Respuesta HTML	El servlet decide el tipo y el contenido de la respuesta.
5	Formulario POST	Un formulario provoca otra petición HTTP.
Final	Aplicación de cuatro rutas	Todas las piezas colaboran en un flujo completo.

1 Del método main a una aplicación web

El modelo anterior

```
public class Programa {
    public static void main(String[] args) {
        System.out.println("Hola");
    }
}
```

Al ejecutar este programa, la máquina virtual de Java busca el método main. Nosotros iniciamos el programa y main marca el punto de entrada. System.out.println escribe en la consola del proceso. No existe navegador, URL, petición HTTP ni servidor web.

El nuevo modelo mental

En una aplicación web no esperamos que una persona ejecute manualmente una clase concreta. Mantenemos Tomcat en ejecución. El navegador envía una petición y Tomcat decide qué código Java debe atenderla. Ese código es un servlet.

```
NAVEGADOR
   |  petición HTTP: GET /ut3-web/hola
   v
TOMCAT
   |  localiza la ruta /hola
   v
HolaServlet.doGet(request, response)
   |  escribe en response
   v
TOMCAT  -- respuesta HTTP -->  NAVEGADOR
```

Pregunta	Programa con main	Aplicación web
¿Quién inicia la ejecución?	Nosotros ejecutamos la clase.	Tomcat ya está iniciado; una petición provoca la llamada.
¿Cuál es el punto de entrada?	main.	El método adecuado del servlet, por ejemplo doGet o doPost.
¿Dónde aparece la salida?	En la consola.	En una respuesta HTTP que interpreta el navegador.
¿Quién crea los objetos principales?	Nuestro código suele usar new.	Tomcat crea el servlet, request y response.

Cambio esencial. No llamaremos a doGet ni crearemos un servlet con new. Tomcat controla el ciclo de vida y realiza esas llamadas cuando llega una petición compatible.

2 Qué ocurre al escribir una URL

Usaremos este ejemplo concreto:

```
http://localhost:8080/ut3-web/hola?nombre=Ana
```

Fragmento	Significado
http	Protocolo: reglas que siguen cliente y servidor para intercambiar mensajes.
localhost	Host: el equipo al que se envía la petición. localhost significa este mismo ordenador.
8080	Puerto: puerta lógica en la que escucha Tomcat.
/ut3-web	Context path: identifica la aplicación desplegada.
/hola	Ruta del servlet dentro de la aplicación.
?nombre=Ana	Query string: datos añadidos a la URL. El parámetro se llama nombre y su valor es Ana.

El recorrido en ocho pasos

1. El usuario confirma la URL. El navegador separa host, puerto, ruta y parámetros.

2. El navegador construye una petición HTTP. En este caso el método es GET porque se está solicitando un recurso mediante la barra de direcciones.

3. La petición llega al puerto 8080, donde Tomcat está escuchando.

4. Tomcat reconoce el context path /ut3-web y entrega la petición a esa aplicación.

5. Dentro de la aplicación, Tomcat busca qué servlet tiene asociada la ruta /hola.

6. Tomcat crea o reutiliza la instancia del servlet y llama a doGet. También proporciona request y response.

7. El servlet consulta request y escribe la salida en response.

8. Tomcat termina la respuesta HTTP y la envía. El navegador muestra el cuerpo recibido.

Petición y respuesta no son lo mismo

La petición viaja del cliente al servidor y describe qué quiere el cliente. La respuesta viaja del servidor al cliente y contiene el resultado. Aunque sucedan en milisegundos, son dos mensajes diferentes.

Mensaje	Dirección	Puede contener
Petición HTTP	Navegador → Tomcat	Método, ruta, parámetros, cabeceras y, en algunos POST, cuerpo.
Respuesta HTTP	Tomcat → navegador	Código de estado, cabeceras, tipo de contenido y cuerpo.

3 Preparar el proyecto web y Tomcat

Qué es Tomcat

Tomcat es el programa servidor que permanece a la espera de peticiones HTTP y sabe ejecutar servlets. Actúa como contenedor: crea las instancias, prepara los objetos de petición y respuesta, asocia rutas con servlets y llama al método correspondiente.

Tomcat no escribe la lógica de nuestra aplicación, no inventa las rutas y no decide qué saludo queremos mostrar. Nosotros programamos esas decisiones. Tampoco es el navegador: recibe mensajes HTTP y devuelve mensajes HTTP.

Estructura mínima con Maven

```
ut3-web/
├─ pom.xml
└─ src/
   └─ main/
      ├─ java/
      │  └─ com/ejemplo/ut3/
      │     └─ HolaServlet.java
      └─ webapp/
         └─ index.html
```

src/main/java contiene código Java. src/main/webapp contiene recursos que Tomcat puede servir directamente, como archivos HTML. El empaquetado war indica que el resultado es una aplicación web desplegable.

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
                             https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <groupId>com.ejemplo</groupId>
    <artifactId>ut3-web</artifactId>
    <version>1.0-SNAPSHOT</version>
    <packaging>war</packaging>

    <properties>
        <maven.compiler.release>17</maven.compiler.release>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    </properties>

    <dependencies>
        <dependency>
            <groupId>jakarta.servlet</groupId>
            <artifactId>jakarta.servlet-api</artifactId>
            <version>6.0.0</version>
            <scope>provided</scope>
        </dependency>
    </dependencies>
</project>
```

La dependencia permite compilar clases como HttpServlet. El scope provided significa que Maven la usa para compilar, pero no la empaqueta: Tomcat ya proporciona esa API al ejecutar la aplicación. Sin la dependencia, el editor y el compilador no reconocerían las clases servlet; si se empaquetara otra copia, podrían aparecer conflictos.

Context path

El context path es el prefijo que identifica la aplicación en Tomcat. Si el artefacto se despliega como ut3-web.war, normalmente será /ut3-web. No forma parte de @WebServlet. La anotación declara la ruta interna; la URL real combina servidor, context path y ruta interna.

```
URL completa = http://localhost:8080 + /ut3-web + /hola
```

4 Mini práctica 1 Primer servlet HolaServlet

Objetivo y resultado esperado

Crear una ruta /hola que ejecute Java y devuelva el texto Hola desde mi primer servlet. Al visitar http://localhost:8080/ut3-web/hola debe aparecer ese texto en el navegador.

Paso 1 Crear la clase

1. Dentro de src/main/java crea el paquete com.ejemplo.ut3.

2. Dentro del paquete crea la clase HolaServlet.

3. Sustituye su contenido por el siguiente código.

```
package com.ejemplo.ut3;

import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

import java.io.IOException;

@WebServlet("/hola")
public class HolaServlet extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest request,
                         HttpServletResponse response)
            throws ServletException, IOException {

        response.getWriter().println("Hola desde mi primer servlet");
    }
}
```

Paso 2 Desplegar y probar

1. Arranca Tomcat desde el entorno de desarrollo.

2. Despliega el artefacto ut3-web:war. Si el entorno ofrece actualización o redeploy, ejecútalo tras cambiar el código.

3. Abre http://localhost:8080/ut3-web/hola.

4. Comprueba que aparece Hola desde mi primer servlet.

Explicación casi línea a línea

package com.ejemplo.ut3

Declara el paquete Java de la clase. Sirve para organizar clases y evitar colisiones de nombres. Lo utiliza el compilador y forma parte del nombre completo com.ejemplo.ut3.HolaServlet. No define ninguna URL.

Los import

Cada import permite escribir el nombre corto de una clase. WebServlet representa la anotación de ruta. HttpServlet es la clase base. HttpServletRequest y HttpServletResponse son los tipos de los objetos que Tomcat entregará. ServletException e IOException describen problemas que el método puede comunicar.

@WebServlet("/hola")

Es una anotación que asocia la ruta interna /hola con esta clase. Tomcat la lee durante el despliegue. Sin esta anotación —y sin una configuración equivalente en web.xml— Tomcat no sabría que /hola debe llegar a HolaServlet. La barra inicial es importante: la ruta se expresa desde la raíz de la aplicación.

> **No confundas.** HolaServlet es el nombre de una clase Java. /hola es una ruta pública. Podrían ser completamente distintos; el navegador solo conoce la URL.

extends HttpServlet

Indica que HolaServlet hereda el comportamiento básico de HttpServlet. Gracias a esa herencia, Tomcat puede tratar nuestra clase como un servlet HTTP y distribuir las peticiones hacia métodos como doGet y doPost. Si fuese una clase Java cualquiera, @Override sobre doGet no tendría el contrato esperado.

Tomcat crea el objeto

Normalmente Tomcat crea una instancia de HolaServlet al necesitarla por primera vez o durante el arranque si está configurada para carga temprana. Después puede reutilizar la misma instancia para muchas peticiones. Por eso no escribimos new HolaServlet() y no guardamos datos de un usuario en atributos del servlet.

@Override

Pide al compilador que compruebe que estamos sobrescribiendo un método heredado. Si escribimos doget, cambiamos los parámetros o cometemos otro error en la firma, el compilador nos avisará. Sin @Override el error podría pasar más desapercibido y Tomcat seguiría ejecutando la implementación heredada.

protected void doGet(...)

doGet atiende peticiones HTTP GET. protected permite que el método sea accesible en el marco de la herencia sin hacerlo público innecesariamente. void significa que no devuelve un valor Java: la salida se escribe en response. Tomcat, no nuestro código, invoca este método.

request y response

request representa la petición ya recibida: ruta, parámetros, cabeceras y otros datos. response representa la respuesta que estamos construyendo: tipo de contenido, estado y cuerpo. Los nombres request y response podrían cambiarse, pero son descriptivos. Los objetos no se crean con new; Tomcat los proporciona para esta petición concreta.

throws ServletException, IOException

La firma permite propagar dos clases de problema. ServletException expresa un fallo relacionado con el procesamiento del servlet. IOException puede aparecer al leer o escribir datos. No significa que siempre vaya a ocurrir un error; significa que, si ocurre y no lo resolvemos aquí, puede comunicarse al contenedor.

response.getWriter().println(...)

getWriter obtiene un escritor de caracteres conectado al cuerpo de la respuesta HTTP. println entrega texto a ese escritor. El texto no se imprime en la consola: viaja en la respuesta hasta el navegador. Tomcat gestiona el cierre normal al terminar el método; no necesitamos cerrar manualmente el escritor en este ejemplo.

5 Reconstruir el flujo de la primera ejecución

```
1  Navegador: GET /ut3-web/hola
2  Tomcat: selecciona la aplicación /ut3-web
3  Tomcat: encuentra @WebServlet("/hola")
4  Tomcat: crea o reutiliza HolaServlet
5  Tomcat: crea request y response
6  Tomcat: llama a doGet(request, response)
7  Servlet: escribe texto con response.getWriter()
8  Tomcat: envía la respuesta HTTP
9  Navegador: muestra el cuerpo recibido
```

La clase no se ejecutó por estar abierta en el editor. Se ejecutó porque la ruta de una petición coincidió con su mapeo. El texto apareció en el navegador porque se escribió en response. Si se hubiera usado System.out.println, el texto habría aparecido en la consola de Tomcat, no como contenido de la página.

Comprobación rápida

- Cambia el mensaje por otro, actualiza el despliegue y recarga la URL.
- Prueba /ut3-web/hola2. Debe aparecer un 404 porque esa ruta no existe.
- Mira la consola de Tomcat al arrancar. Distingue los mensajes del servidor del contenido que recibe el navegador.

6 Mini práctica 2 Una segunda ruta

Objetivo

Comprobar que dos rutas pueden seleccionar dos servlets diferentes y que la URL se decide con @WebServlet, no con el nombre de la clase.

```
package com.ejemplo.ut3;

import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import java.io.IOException;

@WebServlet("/saludo")
public class MensajeDeBienvenida extends HttpServlet {
    @Override
    protected void doGet(HttpServletRequest request,
                         HttpServletResponse response)
            throws ServletException, IOException {
        response.getWriter().println("Bienvenido a la aplicación web");
    }
}
```

1. Crea MensajeDeBienvenida.java en el mismo paquete.

2. Despliega de nuevo la aplicación.

3. Visita /ut3-web/saludo y comprueba el mensaje.

4. Visita /ut3-web/MensajeDeBienvenida. Debe dar 404 salvo que exista un mapeo con ese texto.

Elemento	Lo ve el navegador	Lo usa Tomcat	Lo usa Java
/saludo	Sí, en la URL.	Sí, para elegir el servlet.	Aparece en la anotación.
MensajeDeBienvenida	No.	Sí, como clase gestionada.	Sí, es el nombre de la clase.
/ut3-web	Sí, en la URL.	Sí, para elegir la aplicación.	No se escribe en @WebServlet.

Diagnóstico útil. Si /hola funciona pero /saludo devuelve 404, Tomcat está arrancado y el context path probablemente es correcto. Revisa el mapeo, la compilación y el redeploy del segundo servlet.

7 Mini práctica 3 Recibir un parámetro por la URL

Objetivo

Hacer que /saludo?nombre=Ana produzca Hola, Ana. El dato viajará desde el navegador dentro de la petición y el servlet lo leerá mediante request.

```
http://localhost:8080/ut3-web/saludo?nombre=Ana
```

El signo ? separa la ruta de la query string. nombre es la clave y Ana es el valor. Con varios parámetros se utiliza &: /saludo?nombre=Ana&curso=DWES. El navegador codifica caracteres especiales; Tomcat los analiza y permite consultarlos por nombre.

```
@WebServlet("/saludo")
public class SaludoServlet extends HttpServlet {
    @Override
    protected void doGet(HttpServletRequest request,
                         HttpServletResponse response)
            throws ServletException, IOException {

        String nombre = request.getParameter("nombre");

        if (nombre == null || nombre.isBlank()) {
            nombre = "visitante";
        }

        response.getWriter().println("Hola, " + nombre);
    }
}
```

Seguir el dato línea a línea

1. El navegador envía la cadena nombre=Ana junto con la petición GET.

2. Tomcat interpreta los parámetros y crea HttpServletRequest.

3. request.getParameter("nombre") busca exactamente la clave nombre.

4. El método devuelve un String porque los parámetros HTTP llegan como texto, incluso si parecen números.

5. Si la clave no existe, devuelve null. Si existe sin contenido, puede devolver una cadena vacía.

6. isBlank detecta una cadena vacía o formada solo por espacios; se comprueba null antes para evitar NullPointerException.

7. El servlet concatena el valor y escribe Hola, Ana en response.

8. Tomcat envía la respuesta y el navegador la muestra.

Pruebas obligatorias

URL	Valor obtenido	Respuesta esperada
/saludo?nombre=Ana	"Ana"	Hola, Ana
/saludo?nombre=Luis	"Luis"	Hola, Luis
/saludo	null	Hola, visitante
/saludo?nombre=	cadena vacía	Hola, visitante
/saludo?Nombre=Ana	null	Hola, visitante; Nombre y nombre no coinciden.

Los nombres deben coincidir. getParameter("nombre") no busca cualquier dato parecido. Busca la clave exacta nombre. Las mayúsculas importan.

Un número sigue llegando como texto

```
String edadTexto = request.getParameter("edad");
int edad = Integer.parseInt(edadTexto);
```

La conversión es responsabilidad del programa. Si falta edad o no contiene un entero válido, parseInt fallará. En esta primera aproximación comprobaremos el dato antes de convertirlo; más adelante se diseñará una validación más completa.

8 Mini práctica 4 Varios parámetros y salida HTML

Objetivo

Crear /datos?nombre=Ana&curso=DWES&edad=20 y generar una pequeña página HTML. Aprenderemos que el cuerpo es texto, pero el navegador necesita saber cómo interpretarlo.

```
package com.ejemplo.ut3;

import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import java.io.IOException;
import java.io.PrintWriter;

@WebServlet("/datos")
public class DatosServlet extends HttpServlet {
    @Override
    protected void doGet(HttpServletRequest request,
                         HttpServletResponse response)
            throws ServletException, IOException {

        request.setCharacterEncoding("UTF-8");
        response.setContentType("text/html;charset=UTF-8");

        String nombre = request.getParameter("nombre");
        String curso = request.getParameter("curso");
        String edad = request.getParameter("edad");

        if (nombre == null || nombre.isBlank()) nombre = "Sin indicar";
        if (curso == null || curso.isBlank()) curso = "Sin indicar";
        if (edad == null || edad.isBlank()) edad = "Sin indicar";

        PrintWriter out = response.getWriter();
        out.println("<!DOCTYPE html>");
        out.println("<html lang='es'>");
        out.println("<head><meta charset='UTF-8'><title>Datos</title></head>");
        out.println("<body>");
        out.println("<h1>Datos recibidos</h1>");
        out.println("<p>Nombre: " + nombre + "</p>");
        out.println("<p>Curso: " + curso + "</p>");
        out.println("<p>Edad: " + edad + "</p>");
        out.println("</body></html>");
    }
}
```

Por qué se indica el tipo de contenido

response.setContentType("text/html;charset=UTF-8") añade a la respuesta una cabecera que informa de dos decisiones. text/html indica que el cuerpo contiene HTML. charset=UTF-8 indica cómo convertir caracteres como á, ñ o ü en bytes y cómo interpretarlos. Debe llamarse antes de obtener y usar el escritor para que la codificación se aplique correctamente.

Sin el tipo adecuado, el navegador puede interpretar la respuesta como texto simple o elegir una codificación incorrecta. El HTML podría mostrarse como etiquetas literales o los acentos podrían aparecer dañados.

Qué hace PrintWriter

PrintWriter ofrece métodos para escribir caracteres en el cuerpo de la respuesta. La variable out solo es una referencia cómoda al mismo escritor que antes obteníamos con response.getWriter(). Cada println añade una parte del documento HTML; todas las partes forman un único cuerpo HTTP.

> **Seguridad que veremos después.** Insertar directamente texto enviado por el usuario dentro de HTML puede permitir inyección de marcado o scripts. En este bloque nos centramos en el flujo HTTP; en aplicaciones reales la salida debe escaparse. No uses este patrón sin protección con datos no confiables.

Ejecutar y analizar

1. Despliega DatosServlet.

2. Visita /ut3-web/datos?nombre=Ana&curso=DWES&edad=20.

3. Comprueba el título, el encabezado y los tres párrafos.

4. Quita edad de la URL y comprueba que aparece Sin indicar.

5. Usa Ver código fuente en el navegador y observa que recibió un documento HTML completo.

9 GET y POST desde la práctica

Lo que ya hemos hecho con GET

Al escribir una URL, pulsar un enlace o enviar un formulario con method="get", el navegador realiza normalmente una petición GET. GET se usa para obtener o consultar una representación. Sus parámetros suelen verse en la URL, lo que facilita copiarla, guardarla y repetirla.

Qué cambia con POST

POST también envía una petición, pero los datos de un formulario suelen ir en el cuerpo del mensaje y no en la query string visible. Tomcat reconoce el método POST y llama a doPost en vez de doGet. POST es apropiado cuando la operación envía datos para procesarlos o pretende producir un cambio.

Aspecto	GET	POST
Método del servlet	doGet	doPost
Datos de formulario	Habitualmente en la URL.	Habitualmente en el cuerpo.
Uso típico	Consultar o mostrar.	Enviar para procesar o modificar.
Se puede guardar la URL con los datos	Sí, normalmente.	No de la misma forma.
¿Es seguro por sí mismo?	No.	No. HTTPS protege el transporte; validar sigue siendo obligatorio.
¿Tiene parámetros?	Sí.	Sí. getParameter funciona en ambos casos para formularios normales.

> **POST no significa secreto.** Que un dato no aparezca en la barra de direcciones no lo cifra. Para protegerlo durante el transporte se necesita HTTPS. Tampoco sustituye la validación ni la autorización.

10 Mini práctica 5 Formulario y doPost

Objetivo

Crear un formulario HTML que pida nombre y curso. Al pulsar Enviar, el navegador realizará POST hacia /procesar-saludo y Tomcat ejecutará doPost.

Paso 1 Crear el formulario

Crea src/main/webapp/formulario.html con este contenido:

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Formulario de saludo</title>
</head>
<body>
    <h1>Formulario de saludo</h1>

    <form action="procesar-saludo" method="post">
        <label for="nombre">Nombre:</label>
        <input id="nombre" name="nombre" type="text" required>

        <label for="curso">Curso:</label>
        <input id="curso" name="curso" type="text" required>

        <button type="submit">Enviar</button>
    </form>
</body>
</html>
```

Explicar el formulario línea a línea

form agrupa controles que se enviarán juntos. action="procesar-saludo" indica el destino relativo: desde /ut3-web/formulario.html conduce a /ut3-web/procesar-saludo. method="post" decide el método HTTP y, por tanto, que Tomcat busque doPost.

label describe un campo; for="nombre" apunta al id="nombre" y mejora la accesibilidad. input crea el control. id identifica el elemento dentro del HTML. name="nombre" es la clave HTTP que viajará: es la que debe coincidir con getParameter("nombre"). required pide al navegador que impida un envío vacío, aunque el servidor debe validar igualmente.

button type="submit" inicia el envío del formulario. Al pulsarlo no se llama directamente a Java: el navegador reúne los controles con name, construye una petición POST y la envía a la URL de action.

Paso 2 Crear el servlet

```
package com.ejemplo.ut3;

import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import java.io.IOException;

@WebServlet("/procesar-saludo")
public class ProcesarSaludoServlet extends HttpServlet {
    @Override
    protected void doPost(HttpServletRequest request,
                          HttpServletResponse response)
            throws ServletException, IOException {

        request.setCharacterEncoding("UTF-8");
        response.setContentType("text/html;charset=UTF-8");

        String nombre = request.getParameter("nombre");
        String curso = request.getParameter("curso");

        if (nombre == null || nombre.isBlank()
                || curso == null || curso.isBlank()) {
            response.setStatus(HttpServletResponse.SC_BAD_REQUEST);
            response.getWriter().println("Faltan datos obligatorios");
            return;
        }

        response.getWriter().println("<!DOCTYPE html>");
        response.getWriter().println("<html lang='es'><head>");
        response.getWriter().println("<meta charset='UTF-8'>");
        response.getWriter().println("<title>Resultado</title></head><body>");
        response.getWriter().println("<h1>Saludo recibido</h1>");
        response.getWriter().println("<p>Hola, " + nombre + "</p>");
        response.getWriter().println("<p>Curso: " + curso + "</p>");
        response.getWriter().println("</body></html>");
    }
}
```

Nuevas líneas importantes

doPost tiene el mismo tipo de objetos request y response, pero responde al método POST. request.setCharacterEncoding("UTF-8") debe ejecutarse antes de leer parámetros del cuerpo para interpretar correctamente sus caracteres. response.setStatus cambia el código HTTP a 400 cuando los datos son inválidos. return detiene el método para no escribir también la respuesta de éxito.

Paso 3 Ejecutar

1. Despliega la aplicación.

2. Abre /ut3-web/formulario.html.

3. Escribe Ana y DWES.

4. Pulsa Enviar.

5. Comprueba que la URL final contiene /procesar-saludo pero no muestra los dos parámetros.

6. Comprueba que el contenido procede de doPost.

Flujo completo del formulario

```
formulario.html
   | usuario escribe Ana y DWES
   | submit: action=procesar-saludo, method=post
   v
POST /ut3-web/procesar-saludo
Cuerpo: nombre=Ana&curso=DWES
   v
Tomcat encuentra @WebServlet("/procesar-saludo")
   v
doPost(request, response)
   | request.getParameter("nombre") -> "Ana"
   | escribe HTML en response
   v
Respuesta HTTP 200 con text/html;charset=UTF-8
   v
El navegador representa la página de resultado
```

11 doGet y doPost sin confusiones

Regla de trabajo para este bloque

- Usa doGet para mostrar o consultar información sin pretender cambiar el estado del servidor.
- Usa doPost cuando un formulario envía datos para ser procesados o cuando la operación pretende crear o cambiar algo.
- No elijas POST solo para ocultar parámetros y no elijas GET por ser más sencillo.
- El método HTTP determina qué método del servlet llama Tomcat. La ruta determina qué servlet recibe la petición. Son decisiones distintas.

Misma ruta, métodos distintos

```
@WebServlet("/perfil")
public class PerfilServlet extends HttpServlet {
    @Override
    protected void doGet(HttpServletRequest request,
                         HttpServletResponse response) throws IOException {
        response.getWriter().println("Mostrar formulario");
    }

    @Override
    protected void doPost(HttpServletRequest request,
                          HttpServletResponse response) throws IOException {
        response.getWriter().println("Procesar formulario");
    }
}
```

GET /perfil y POST /perfil tienen la misma ruta, pero no son la misma petición. Tomcat selecciona el mismo servlet por la ruta y después el método adecuado por el verbo HTTP.

Errores frecuentes

- Abrir una ruta POST en la barra de direcciones: la barra envía GET y doPost no se ejecuta.
- Poner method="post" pero implementar solo doGet: el contenedor suele responder 405 Method Not Allowed.
- Creer que getParameter solo sirve para GET: también lee campos de formularios POST normales.
- Hacer que un GET borre o modifique datos: recargar, guardar o rastrear la URL podría repetir la acción.
- Pensar que POST cifra datos: solo HTTPS cifra el transporte.

12 Request y response son objetos de Tomcat

HttpServletRequest

Es la representación Java de una petición HTTP concreta. Tomcat la crea al recibir el mensaje y la rellena con la información analizada. El servlet la usa durante esa petición. No debemos almacenarla para utilizarla más tarde.

Necesidad	Operación
Leer un parámetro	request.getParameter("nombre")
Saber el método HTTP	request.getMethod()
Consultar la ruta solicitada	request.getRequestURI()
Leer una cabecera	request.getHeader("User-Agent")
Interpretar el cuerpo con UTF-8	request.setCharacterEncoding("UTF-8") antes de getParameter

HttpServletResponse

Es el objeto mutable con el que construimos la respuesta. Al principio está esencialmente vacía. El servlet decide el estado, las cabeceras y el cuerpo. Tomcat convierte esas decisiones en un mensaje HTTP y lo envía por la conexión.

Necesidad	Operación
Indicar HTML y codificación	response.setContentType("text/html;charset=UTF-8")
Escribir el cuerpo	response.getWriter().println(...)
Indicar petición incorrecta	response.setStatus(HttpServletResponse.SC_BAD_REQUEST)
Enviar un error	response.sendError(404, "No encontrado")

Orden importante. Configura tipo y codificación antes de escribir el cuerpo. Cuando la respuesta ya se ha enviado parcialmente, cambiar cabeceras o estado puede no tener efecto.

13 Quién crea el servlet y cuándo

Al desplegar la aplicación, Tomcat descubre las clases anotadas. Puede crear un servlet durante el arranque o cuando llega la primera petición. Habitualmente mantiene una instancia y la reutiliza para atender muchas peticiones, incluso simultáneas.

```
Tomcat
  crea UNA instancia gestionada de HolaServlet
       |
       +-- petición A -> doGet(requestA, responseA)
       |
       +-- petición B -> doGet(requestB, responseB)
```

Cada petición tiene sus propios request y response, pero puede pasar por la misma instancia del servlet. Por eso las variables locales dentro de doGet o doPost son adecuadas para datos de la petición. Un atributo de instancia como private String nombreActual sería compartido y podría mezclar usuarios.

```
// Evitar: estado de una petición guardado en el servlet
private String nombreActual;

// Preferir: variable local para esta llamada
String nombre = request.getParameter("nombre");
```

Por qué no hacemos new HolaServlet()

Un objeto creado manualmente no está registrado como el componente que Tomcat administra para la ruta, y no tendría por sí solo una petición ni una respuesta reales. La anotación y el contenedor resuelven el ciclo de vida. Nuestro trabajo es declarar el mapeo e implementar el comportamiento.

14 Guía de diagnóstico de errores

Síntoma	Causa probable	Cómo localizarlo
No se puede conectar	Tomcat no está arrancado o el puerto no es 8080.	Comprueba el estado del servidor y su registro de inicio; confirma el puerto.
404 Not Found	La aplicación o la ruta no coinciden.	Separa URL en host, puerto, context path y @WebServlet. Prueba primero una ruta conocida.
404 tras añadir un servlet	El cambio no se ha compilado o desplegado.	Haz build y redeploy; busca errores de compilación y mensajes de despliegue.
405 Method Not Allowed	La ruta existe, pero falta doGet o doPost para el método enviado.	Comprueba method del formulario y qué método sobrescribe el servlet.
Parámetro null	El name o la clave de getParameter no coinciden.	Compara carácter por carácter name="nombre" y getParameter("nombre").
Se ejecuta el método heredado	Firma incorrecta o @Override ausente.	Añade @Override y usa exactamente los tipos y el nombre doGet/doPost.
Acentos dañados	Codificación no establecida a tiempo.	Configura request antes de leer y response antes de escribir.
Las etiquetas se ven como texto	Content-Type incorrecto o ausente.	Usa text/html;charset=UTF-8 antes de getWriter.
Cambio de mensaje no aparece	Despliegue antiguo o caché.	Redeploy y recarga; confirma la consola y la URL exacta.
Ruta aparentemente correcta falla	Context path distinto al supuesto.	Consulta la configuración del artefacto; no copies /ut3-web a ciegas.

Método de diagnóstico en orden

1. ¿Tomcat está arrancado y escucha en el puerto correcto?

2. ¿La aplicación se ha desplegado sin errores?

3. ¿El context path de la URL es el real?

4. ¿La ruta coincide exactamente con @WebServlet?

5. ¿El método HTTP coincide con doGet o doPost?

6. ¿Los nombres de parámetros coinciden?

7. ¿El error aparece en la consola de Tomcat? Lee la primera causa útil, no solo la última línea.

> **404 y 500 significan cosas distintas.** 404 indica que no se encontró el recurso solicitado. 500 indica que el servidor encontró la ruta, pero ocurrió un fallo al procesarla. En un 500 revisa la excepción de la consola.

15 Práctica final guiada

Aplicación Navegación básica

Construirás una pequeña aplicación con cuatro puntos de entrada: /inicio, /saludo?nombre=..., /datos y un formulario POST. No usa JSP, MVC, base de datos ni sesiones. Su finalidad es consolidar rutas, parámetros, métodos y flujo HTTP.

Resultado que debe existir

Recurso	Método	Resultado
/inicio	GET	Página HTML con enlaces al resto.
/saludo?nombre=Ana	GET	Saludo personalizado y valor por defecto.
/datos?nombre=Ana&ciudad=León	GET	Página con dos datos recibidos.
/registro.html	GET estático	Formulario con nombre y correo.
/procesar-registro	POST	Página de confirmación o error 400.

Paso 1 Crear InicioServlet

```
@WebServlet("/inicio")
public class InicioServlet extends HttpServlet {
    @Override
    protected void doGet(HttpServletRequest request,
                         HttpServletResponse response) throws IOException {
        response.setContentType("text/html;charset=UTF-8");
        response.getWriter().println("<!DOCTYPE html>");
        response.getWriter().println("<html lang='es'><head><meta charset='UTF-8'>");
        response.getWriter().println("<title>Inicio</title></head><body>");
        response.getWriter().println("<h1>Aplicación de prácticas</h1>");
        response.getWriter().println("<ul>");
        response.getWriter().println("<li><a href='saludo?nombre=Ana'>Saludo</a></li>");
        response.getWriter().println("<li><a
href='datos?nombre=Ana&ciudad=León'>Datos</a></li>");
        response.getWriter().println("<li><a href='registro.html'>Registro</a></li>");
        response.getWriter().println("</ul></body></html>");
    }
}
```

Los href son relativos al directorio actual. Cuando estamos en /ut3-web/inicio, saludo?nombre=Ana resuelve a /ut3-web/saludo?nombre=Ana. Cada clic crea una nueva petición GET; no es una llamada directa entre clases Java.

Paso 2 Crear SaludoServlet

```
@WebServlet("/saludo")
public class SaludoServlet extends HttpServlet {
    @Override
    protected void doGet(HttpServletRequest request,
                         HttpServletResponse response) throws IOException {
        response.setContentType("text/html;charset=UTF-8");
        String nombre = request.getParameter("nombre");
        if (nombre == null || nombre.isBlank()) nombre = "visitante";

        response.getWriter().println("<!DOCTYPE html>");
        response.getWriter().println("<html lang='es'><head><meta charset='UTF-8'>");
        response.getWriter().println("<title>Saludo</title></head><body>");
        response.getWriter().println("<h1>Hola, " + nombre + "</h1>");
        response.getWriter().println("<a href='inicio'>Volver</a>");
        response.getWriter().println("</body></html>");
    }
}
```

Prueba con nombre, sin nombre y con nombre vacío. Deben funcionar las tres variantes sin lanzar excepciones.

Paso 3 Crear DatosServlet

Adapta la mini práctica 4 para leer nombre y ciudad. Ambos son String. Si falta cualquiera, muestra Sin indicar. Incluye un enlace relativo href="inicio" para volver. Antes de continuar, verifica /datos?nombre=Ana&ciudad=León.

Paso 4 Crear registro.html

```
<!DOCTYPE html>
<html lang="es">
<head><meta charset="UTF-8"><title>Registro</title></head>
<body>
    <h1>Registro básico</h1>
    <form action="procesar-registro" method="post">
        <p>
            <label for="nombre">Nombre:</label>
            <input id="nombre" name="nombre" type="text" required>
        </p>
        <p>
            <label for="correo">Correo:</label>
            <input id="correo" name="correo" type="email" required>
        </p>
        <button type="submit">Registrar</button>
    </form>
    <a href="inicio">Volver</a>
</body>
</html>
```

El navegador puede validar required y type="email", pero esas comprobaciones pueden evitarse. El servlet también debe comprobar que los datos existen.

Paso 5 Crear ProcesarRegistroServlet

```
@WebServlet("/procesar-registro")
public class ProcesarRegistroServlet extends HttpServlet {
    @Override
    protected void doPost(HttpServletRequest request,
                          HttpServletResponse response) throws IOException {
        request.setCharacterEncoding("UTF-8");
        response.setContentType("text/html;charset=UTF-8");

        String nombre = request.getParameter("nombre");
        String correo = request.getParameter("correo");

        if (nombre == null || nombre.isBlank()
                || correo == null || correo.isBlank()) {
            response.setStatus(HttpServletResponse.SC_BAD_REQUEST);
            response.getWriter().println("<h1>Faltan datos</h1>");
            return;
        }

        response.getWriter().println("<!DOCTYPE html>");
        response.getWriter().println("<html lang='es'><head><meta charset='UTF-8'>");
        response.getWriter().println("<title>Confirmación</title></head><body>");
        response.getWriter().println("<h1>Registro recibido</h1>");
        response.getWriter().println("<p>Nombre: " + nombre + "</p>");
        response.getWriter().println("<p>Correo: " + correo + "</p>");
        response.getWriter().println("<a href='inicio'>Volver al inicio</a>");
        response.getWriter().println("</body></html>");
    }
}
```

Paso 6 Prueba sistemática

1. Arranca Tomcat y despliega la aplicación.

2. Abre /inicio y visita los tres enlaces.

3. En /saludo cambia manualmente el parámetro.

4. En /datos elimina uno de los parámetros.

5. Envía el formulario con valores válidos.

6. Intenta abrir /procesar-registro desde la barra. Explica por qué no equivale a enviar el formulario.

7. Corrige cualquier diferencia antes de ampliar la aplicación.

Reconstrucción final del envío POST

1. El navegador obtiene registro.html mediante GET.

2. El usuario rellena controles; todavía no se ha ejecutado el servlet POST.

3. Submit lee los controles que tienen name.

4. El navegador envía POST /ut3-web/procesar-registro con los valores en el cuerpo.

5. Tomcat elige la aplicación, luego la ruta y finalmente doPost.

6. El servlet lee request, valida, configura response y escribe HTML.

7. Tomcat envía el estado, las cabeceras y el cuerpo.

8. El navegador sustituye la página del formulario por la respuesta.

Reto individual

Añade un campo ciudad al formulario y a la confirmación. Después añade una nueva ruta GET /resumen que reciba modulo y horas mediante query string. Debe mostrar valores por defecto cuando falten. Antes de programar, escribe la URL de prueba y dibuja el recorrido navegador → Tomcat → servlet → respuesta.

- La clave name del campo debe coincidir con getParameter.
- La respuesta debe declarar UTF-8.
- No guardes los datos en atributos del servlet.
- Incluye un enlace para volver a /inicio.
- Prueba al menos un caso válido, uno vacío y uno con acentos.

Criterios de comprobación

Comprobación	Conseguido
Distingo context path y ruta del servlet.	□
Sé qué método HTTP provoca cada acción.	□
Puedo localizar dónde viaja cada parámetro.	□
Configuro el tipo de respuesta antes de escribir.	□
Puedo explicar quién crea servlet, request y response.	□
Diagnostico 404, 405 y parámetros null.	□

16 Mapa mental del bloque

```
USUARIO
  escribe URL o envía formulario
            |
            v
NAVEGADOR  construye PETICIÓN HTTP
  método: GET o POST
  destino: host + puerto + context path + ruta
  datos: query string o cuerpo
            |
            v
TOMCAT
  1. recibe la petición
  2. selecciona la aplicación por el context path
  3. selecciona el servlet por @WebServlet
  4. crea/reutiliza el servlet
  5. crea request y response
  6. llama a doGet o doPost
            |
            v
SERVLET
  lee request.getParameter(...)
  aplica decisiones y validaciones
  configura response
  escribe el cuerpo
            |
            v
TOMCAT  envía RESPUESTA HTTP
  estado + cabeceras + cuerpo
            |
            v
NAVEGADOR  interpreta texto o HTML y lo muestra
```

Ideas que deben quedar claras

- Una URL no ejecuta una clase por su nombre: provoca una petición HTTP cuya ruta Tomcat asocia a un servlet.
- El context path identifica la aplicación; @WebServlet define una ruta dentro de ella.
- Tomcat crea y administra el servlet. También proporciona request y response en cada llamada.
- request contiene lo que llegó; response contiene lo que queremos devolver.
- GET y POST son métodos HTTP. La ruta selecciona el servlet y el método selecciona doGet o doPost.
- Los parámetros HTTP se obtienen como String y pueden faltar.
- El tipo y la codificación de la respuesta deben declararse antes de escribir.
- Un 404 suele señalar ruta o despliegue; un 405, método no admitido; un 500, fallo durante la ejecución.
- Cada prueba debe terminar explicando el recorrido completo de la información.

Camino posterior

En los siguientes bloques separaremos mejor la generación de vistas y la lógica, y conectaremos la aplicación con otras capas. Antes de avanzar, este flujo debe poder explicarse sin memorizar código: quién envía, quién recibe, qué objeto contiene cada dato y qué respuesta vuelve al navegador.